Graph analysis with an explainable traversal

Treasure-Island Bridge Safety Map

Finding critical connections and explaining why they matter

A local React and TypeScript application

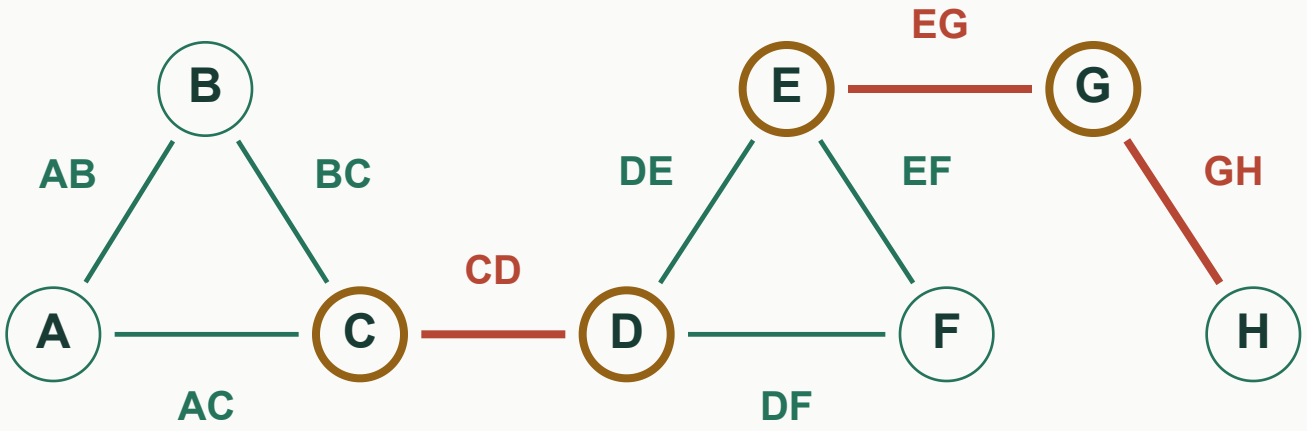
3 November 2026

The problem I solved

A single bridge or island closure can separate the festival

8 islands
9 connections

Two loops provide alternate routes.
CD, EG, and GH expose branches.



Red edges: critical bridges

Amber nodes: articulation islands

Engine limits
1–12 nodes and 0–20 edges

My five-phase implementation plan

I verified the calculation before adding replay and presentation

Phase	Working result	Passing tests
1	Graph models and complete validation	91
2	Deterministic Tarjan analysis	149
3	Typed events and frozen snapshots	190
4	Pure replay and injected controller	261
5	Map, proof panel, table, and summary	293

Later additions: reliability, animation, printing, and exploration. Current suite: 337 tests.

Why I chose this stack

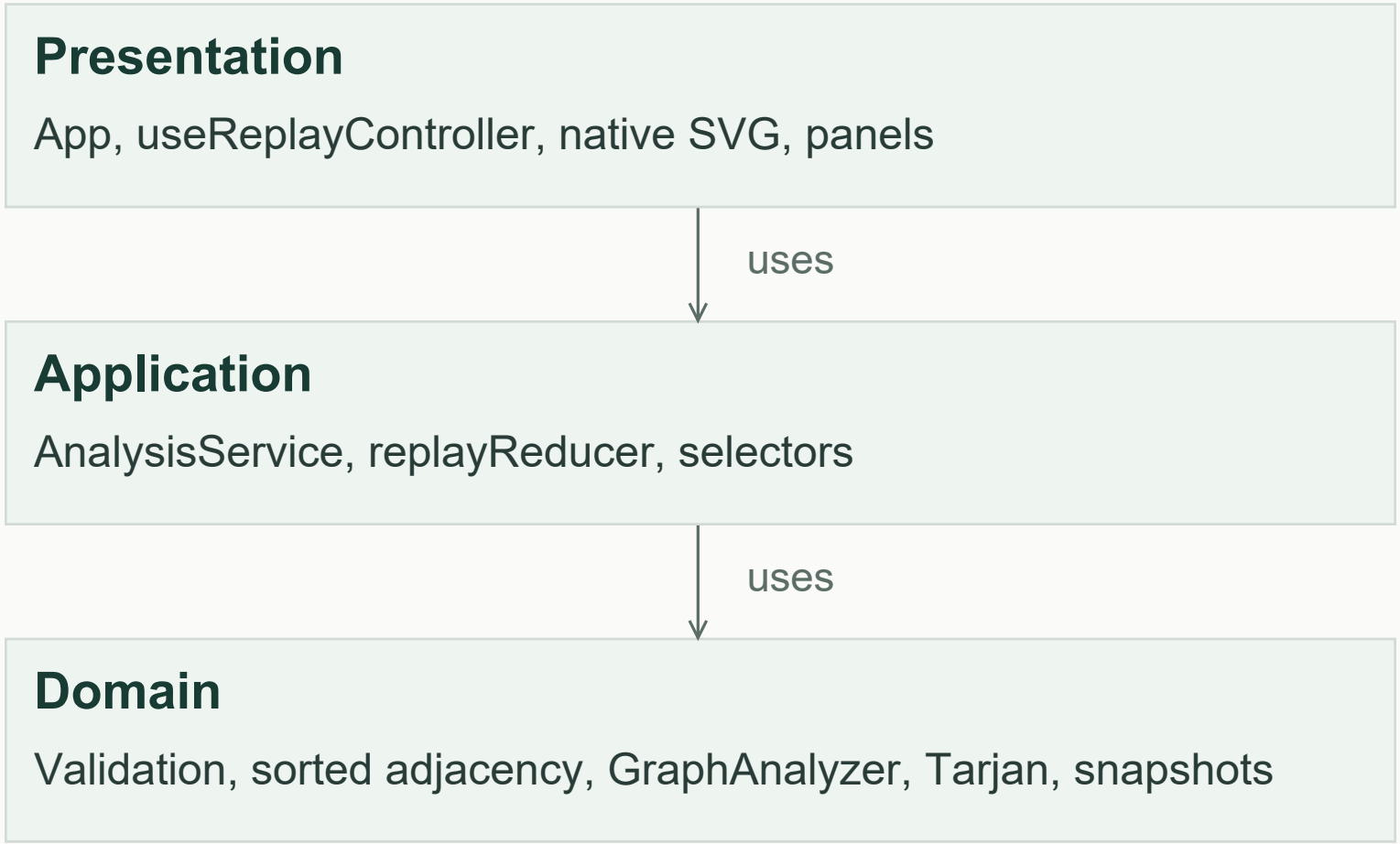
The complete analysis runs locally in the browser

Area	Choice	Reason
Interface	React and TypeScript	Typed components and contracts
Build and styling	Vite, Tailwind CSS, project CSS	Fast local workflow and a small UI stack
Map and replay	Native SVG and useReducer	Exact drawing and explicit transitions
Verification	Vitest, Testing Library, jsdom, Oxlint	Pure tests, DOM checks, and build quality
Delivery	GitHub Actions and Render static site	Repeatable checks and a public demo

All analysis runs in the browser. No backend, database, or real map service.

Architecture and responsibilities

Every panel receives the same selected ReplayView



Load

Validate and analyze once

Replay

Move through saved events

All panels

Render the same snapshot

Patterns used in the implementation

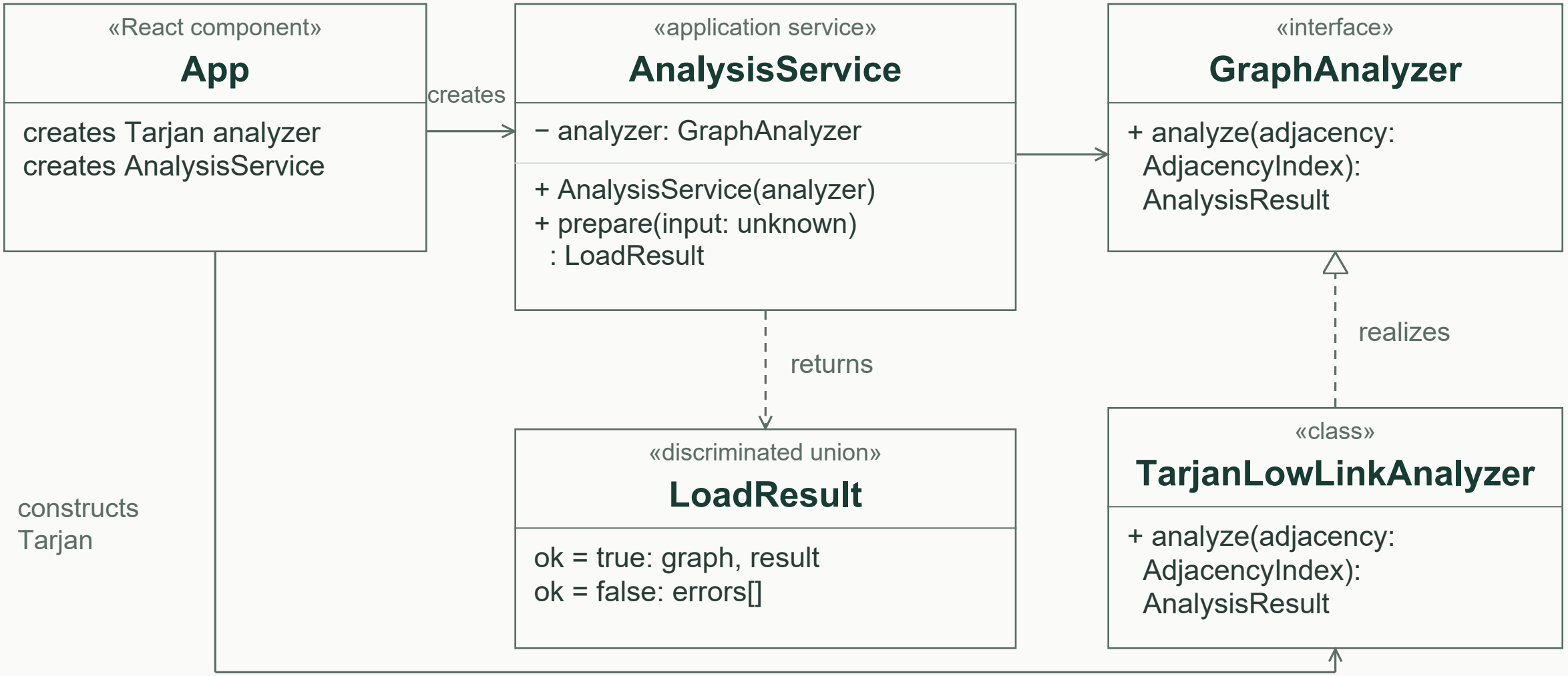
Each pattern has a specific role in the code

Pattern or mechanism	Where I use it	What it gives me
Strategy	GraphAnalyzer and TarjanLowLinkAnalyzer	A replaceable analysis contract
Constructor injection	AnalysisService(analyzer), wired in App	Isolated orchestration and test doubles
Reducer	replayReducer(state, action)	Explicit and testable state transitions
Immutable snapshot history	ReplayEvent.snapshot	Stable explanations without rerunning DFS

The factory is a simple function that returns fresh demo values.

UML: analysis and dependency injection

The service depends on GraphAnalyzer, while App selects Tarjan



Phase 1: validate the graph

I reject invalid input before constructing adjacency or running DFS

Complete input checks

IDs, coordinates, endpoint references, limits, and duplicate connections

Structured failure

Return relevant error paths before adjacency construction or DFS

Independent demo data

Fresh graph values protect Reset from caller mutation

91 passing tests

Build and lint passed

Phase 2: deterministic Tarjan analysis

Sorted roots and neighbors make the traversal repeatable

Discovery starts at zero.
One clock spans all components.

Tree child v
low[u] = min(low[u], low[v])

Visited non-parent neighbor w
low[u] = min(low[u], discovery[w])

Bridge
low[v] > discovery[u]

Non-root articulation
low[v] ≥ discovery[u]

Root articulation
At least two DFS-tree children

58 new tests, 149 total

Phase 3: events and immutable snapshots

I capture each explanation during the existing DFS traversal

ReplayEvent

index, type, focus, message, typed details

owns →

TraversalSnapshot

Node and edge states, components, critical IDs

After child completion

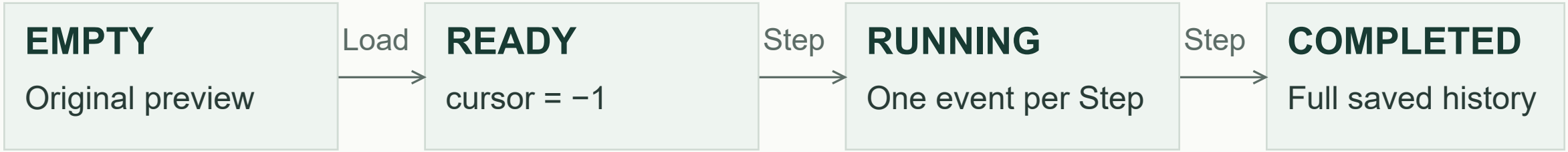
RETURN_CHILD → CLASSIFY_EDGE → new ARTICULATION

Copy nested data, then freeze it. Earlier snapshots cannot change.

41 new tests, 190 total

Phase 4: predictable replay

I move an event cursor instead of rerunning the algorithm



Run to End selects the final event from READY or RUNNING.

LOAD_FAILURE

ERROR clears every earlier visible result together

RESET

Restore the independently owned original graph

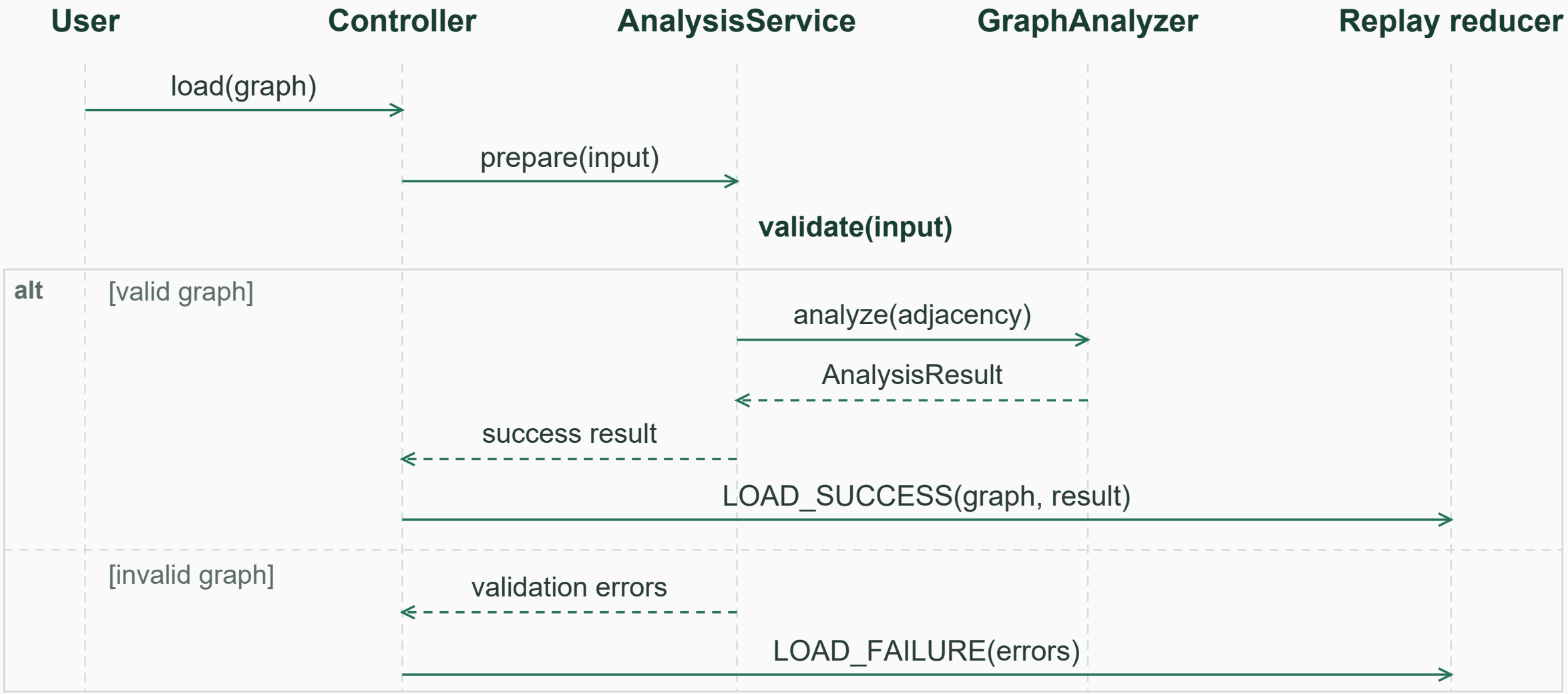
Step, Run, Reset, rerender

Never execute DFS

71 new tests, 261 total

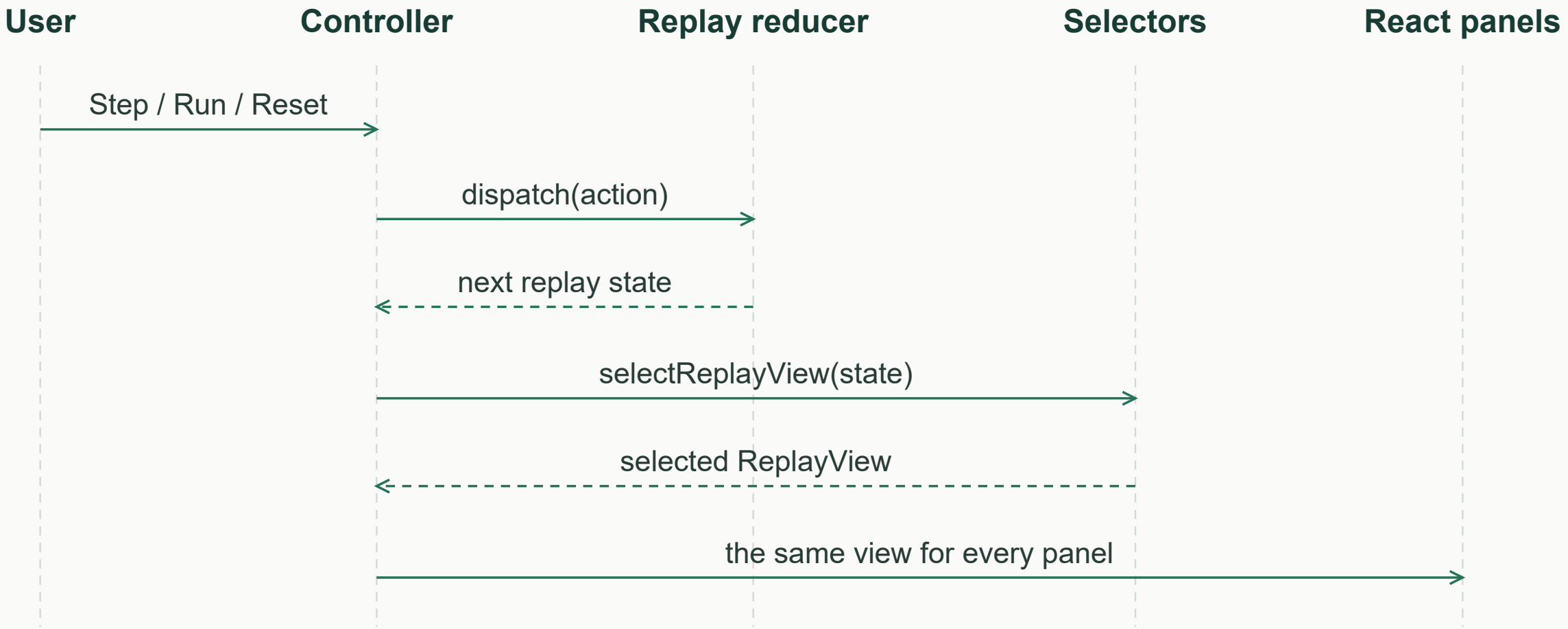
Sequence: loading a graph

Validation gates analysis and failure clears the current replay



Sequence: Step, Run to End, and Reset

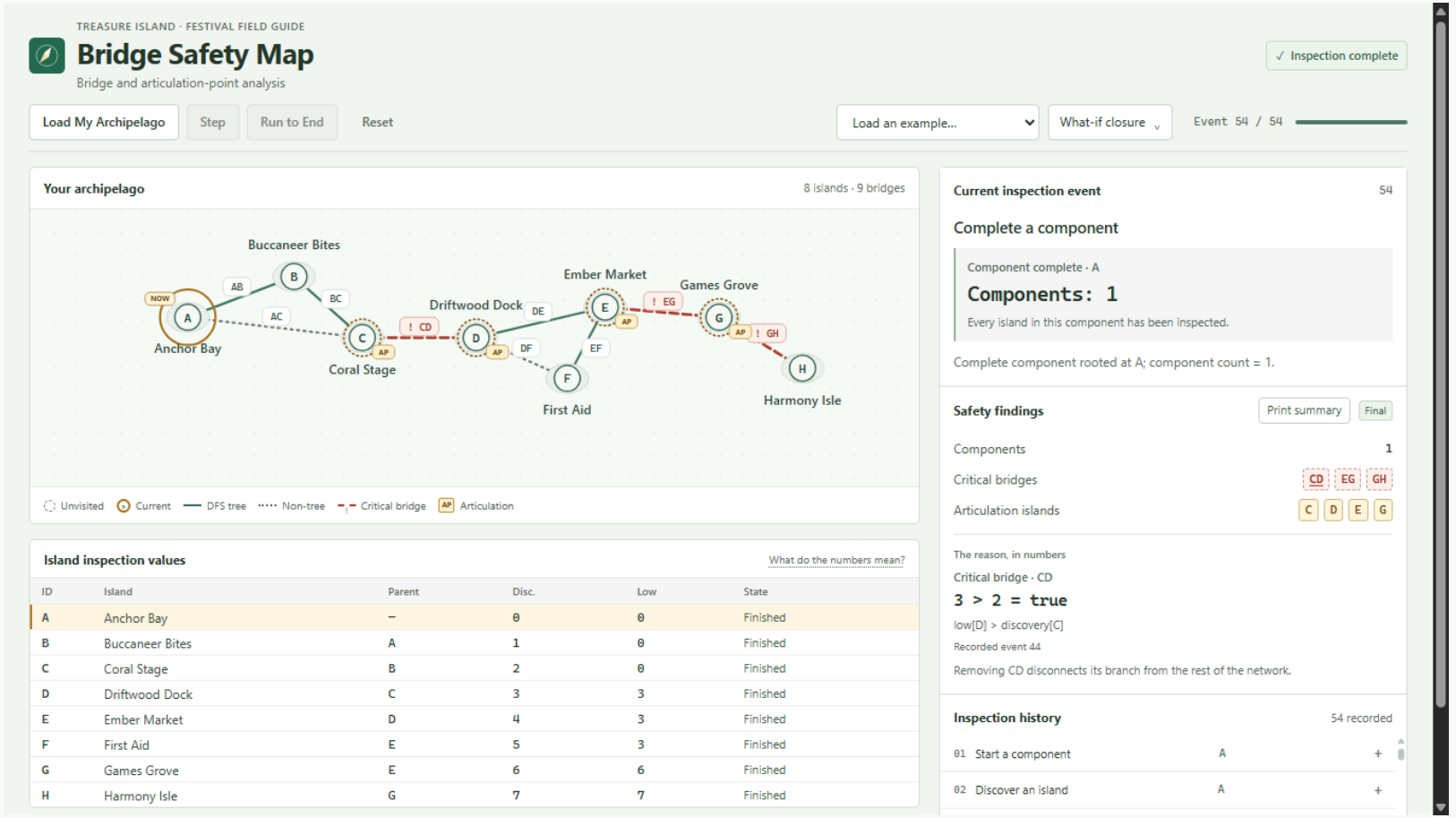
Replay selects a saved snapshot and updates one view



READY: initialSnapshot. After Step: events[cursor].snapshot.

Phase 5: the synchronized interface

The map, values, proof, history, and findings stay at the same event



32 new tests, 293 total at Phase 5

DE and D: the equality case

The same low value answers two different removal questions

21

Return F to E

low[E] = min(4, 3)
4 becomes 3

39

Classify DE

3 > 3 = false
DE is not a bridge

40

Mark D

3 ≥ 3 = true
D is articulation

Removing DE preserves the loop. Removing D separates the surviving islands.

How I used AI during development

I supplied contracts and checked the generated behavior

Actual Phase 3 request

“Generate events during the existing DFS traversal.”
“Copy and freeze nested records and arrays.”

Actual Phase 4 request

“Step, Run to End, Reset, and ordinary rerenders must never execute DFS.”

Checks: independent oracle, exact traces, injected spies, and regression gates

How I verified correctness

The current regression suite has 337 passing tests

337

passing tests in 17 files

Layer	Evidence
Graph correctness	Independent removal oracle over 1,099 graphs
Replay history	Exact traces and 75 generated-graph histories
Application and UI	Transitions, errors, Reset, all 54 demo events
Production	Ubuntu CI, desktop/mobile Chrome, print media

Locked installation, production build, and lint also pass in hosted CI.

What changed during development

I kept the phase order and corrected issues as evidence appeared

Observed issue or refinement	Change and verification
Runtime data can violate Graph types	Validate unknown before creating a typed graph
Accessible names and label collisions	Correct associations and layout, then repeat browser checks
54-event tests exceeded five seconds	Scope queries, use two workers, bound only two tests at 20 s
Optional and exploration scope	Add after the five-phase baseline and retain regression gates

The architecture stayed stable through these corrections.

Optional features and graph exploration

The original inspection stays intact while I explore a closure

Gentle replay animation

CSS transitions respect reduced motion

Printable safety summary

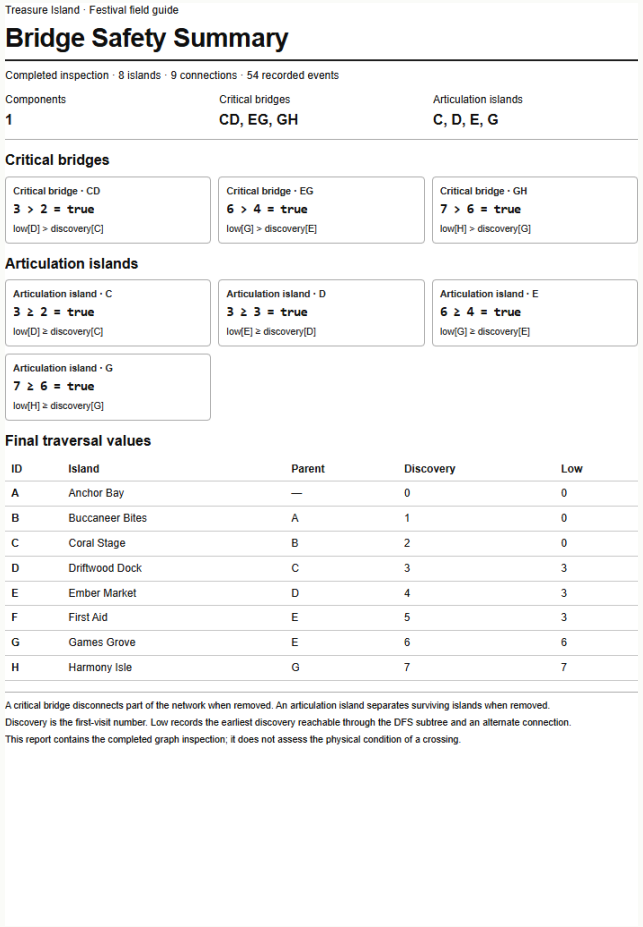
Final table and proofs, with no change to replay

Examples and what-if closure

Cycle, root, disconnected graphs, and closures

treasure-island-map.onrender.com

github.com/sonisiiuu7/treasure-island-map



Treasure Island · Festival field guide

Bridge Safety Summary

Completed inspection · 8 islands · 9 connections · 54 recorded events

Components	Critical bridges	Articulation islands
1	CD, EG, GH	C, D, E, G

Critical bridges

Critical bridge · CD
`3 > 2 = true`
low[D] > discovery[C]

Critical bridge · EG
`6 > 4 = true`
low[G] > discovery[E]

Critical bridge · GH
`7 > 6 = true`
low[H] > discovery[G]

Articulation islands

Articulation island · C
`3 ≥ 2 = true`
low[D] ≥ discovery[C]

Articulation island · D
`3 ≥ 3 = true`
low[E] ≥ discovery[D]

Articulation island · E
`6 ≥ 4 = true`
low[G] ≥ discovery[E]

Articulation island · G
`7 ≥ 6 = true`
low[H] ≥ discovery[G]

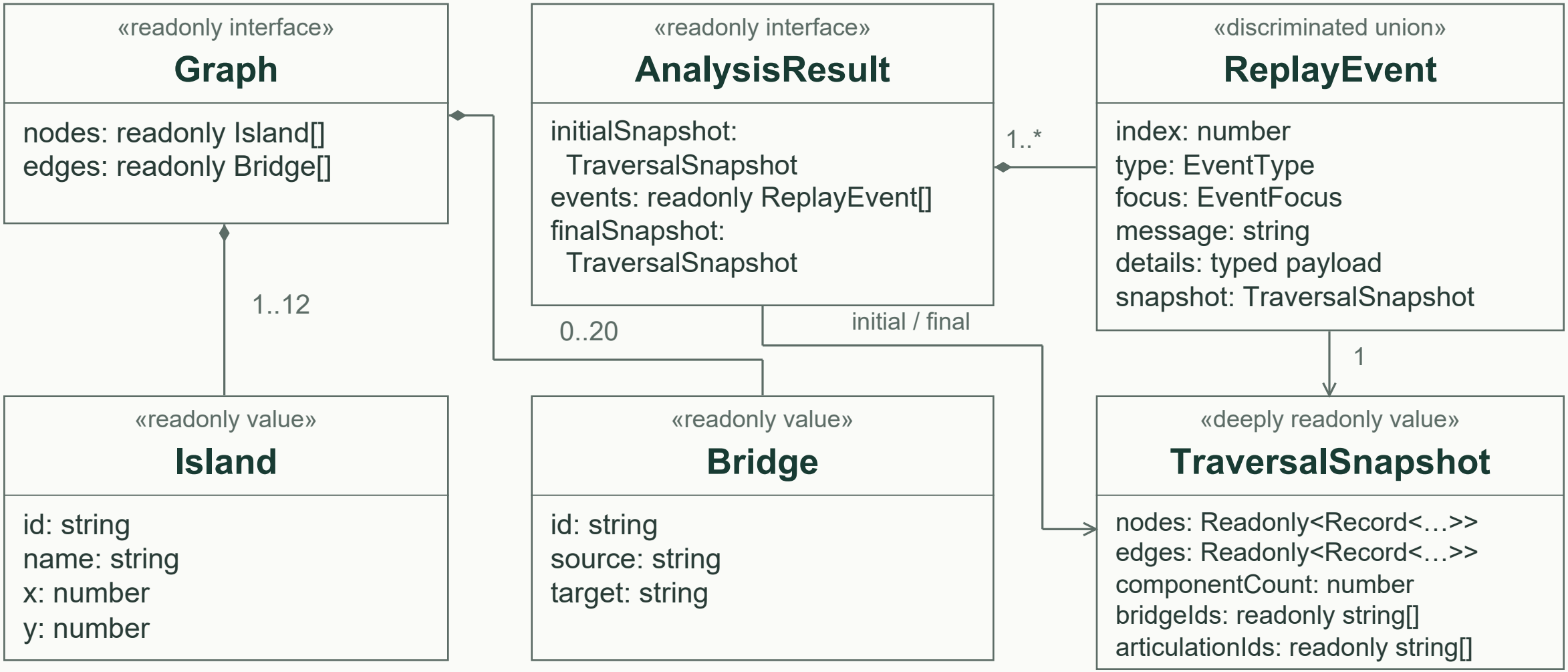
Final traversal values

ID	Island	Parent	Discovery	Low
A	Anchor Bay	—	0	0
B	Buccaneer Bites	A	1	0
C	Coral Stage	B	2	0
D	Driftwood Dock	C	3	3
E	Ember Market	D	4	3
F	First Aid	E	5	3
G	Games Grove	E	6	6
H	Harmony Isle	G	7	7

A critical bridge disconnects part of the network when removed. An articulation island separates surviving islands when removed.
Discovery is the first-visit number. Low records the earliest discovery reachable through the DFS subtree and an alternate connection.
This report contains the completed graph inspection; it does not assess the physical condition of a crossing.

UML: graph and replay models

Readonly value models describe input and historical state



Independent reference values

I compare calculated output with a separately derived table

ID	Parent	Disc.	Low
A	null	0	0
B	A	1	0
C	B	2	0
D	C	3	3
E	D	4	3
F	E	5	3
G	E	6	6
H	G	7	7

Sorted neighbor IDs

A: B, C
B: A, C
C: A, B, D
D: C, E, F
E: D, F, G
F: D, E
G: E, H
H: G

1 component **Bridges CD, EG, GH** **Articulation C, D, E, G**

SOLID principles in the code

I apply the principles at useful boundaries

Principle	Concrete application
Single Responsibility	Validation, adjacency, analysis, replay, and rendering are separate
Open/Closed	AnalysisService accepts a compatible analyzer implementation
Liskov Substitution	A replacement preserves results, order, events, and ownership
Interface Segregation	GraphAnalyzer exposes analyze; the hook needs only prepare
Dependency Inversion	AnalysisService depends on GraphAnalyzer, with App wiring Tarjan

Small helpers remain pure functions where an abstraction adds no useful boundary.

Trade-offs and performance

Full snapshots simplify replay within the bounded graph size

Choice	Benefit	Cost or boundary
Strategy and constructor injection	Testable analysis boundary	Substitutes must preserve semantics
Pure reducer and one selected view	Predictable replay and synchronized panels	Cursor bounds still need runtime rules
Copied and frozen full snapshots	Simple independent historical views	O(K(V+E)) stored data plus copy work
Recursive DFS and native SVG	Small implementation for a bounded graph	Revisit recursion and labels at larger scale

DFS core: O(V+E). Sorting and the stored replay history add separate costs.

Edge cases I tested

The checks cover false outcomes and lifecycle recovery

Case	Required result or invariant
Cycle only	No bridge or articulation
Root with two DFS-tree children	Root is articulation. Degree alone is insufficient.
Disconnected and isolated nodes	Sorted roots and one discovery clock
Visited connection and equality	Use discovery[w], strict >, non-root ≥
Invalid coordinate / duplicate node	Identify errors and clear all stale output
Edge inspections and child completion	Both directions plus return/classify/articulate order
Reset and repeated runs	Original preview and identical immutable histories

A small live extension

Proposed: an event-type filter in the presentation layer

Example user request

“Let me show only one event type in the recorded history.”

Precise instruction for the AI assistant

Add an accessible filter to EventHistory. Filter only reached events.
Preserve cursor, snapshot, progress, event numbers, and analysis-call count.
Clear it on Reset. Test keyboard use, empty matches, and no future events.

Focused tests first, then demonstrate the change and run the regression gate.